

Zero to AI Agent

Build Your Own Automation Army

No Code. No Team. No Excuses.

Jeff — ErgeneAI

June 2026

Battle-tested on a production server since May 2026

Contents

Chapter 1: Introduction — Why AI Agents Changed Everything

Chapter 2: Your First AI Agent

Chapter 3: Powering Up — MCP Servers

Chapter 4: Building Skills — Your Agent's Playbook

Chapter 5: n8n Integration — Workflow Automation

Chapter 6: Monitoring — Watch the World While You Sleep

Chapter 7: Autonomous Loops — Set It and Forget It

Chapter 8: Production Deployment

Chapter 9: Scaling — From Solo to Agency

Chapter 10: The AI Agency Business Model

<h1>Chapter 1: Introduction 🤖🤖🤖 Why AI Agents Changed Everything</h1>

<h2>The Shift From Chat to Action</h2>

You've used ChatGPT, Claude, or Gemini. You ask a question, it answers. That's `_zero_` leverage.

The real leverage comes when AI stops `*talking*` and starts `*doing*`.

An `**AI agent**` isn't just a chatbot. It's an autonomous system that:

- Reads files on your server
- Runs shell commands
- Scrapes websites
- Sends Telegram messages
- Checks your database
- Monitors your competitors
- Sells your products

It's a digital employee that works 24/7, costs cents per hour, and never complains about overtime.

<h2>Why This Book?</h2>

I run a one-person operation called `**ErgeneAI**`. My server runs:

- `**Hermes Agent**` 000 my autonomous AI assistant
- `**n8n**` 000 workflow automation (34 active workflows)
- `**Agent Reach**` 000 multi-platform monitoring
- `**NotebookLM**` 000 document intelligence
- `**Ollama**` 000 local LLMs
- `**PostgreSQL**` 000 vector database for memory

This stack replaces what would be a team of 5 people. It costs me ~\$50/month in server + API costs.

I wrote this book because I couldn't find a practical, step-by-step guide to building this stack.

1. Theoretical fluff ("AI will change everything")
2. Overly technical (academic papers on agent architectures)
3. Platform-specific vendor lock-in

This book is different. It's `**practical**`, `**battle-tested**`, and `**platform-agnostic**`.

<h2>What You'll Build</h2>

By the end of this book, you'll have:

1. **A fully autonomous AI agent** 000 Hermes-compatible, running on your own server
2. **Workflow automation** 000 n8n pipelines that handle real business tasks
3. **Monitoring & intelligence** 000 systems that watch markets, competitors, and opportunities
4. **Revenue-ready setup** 000 a service you can sell to small businesses

<h2>What You Need</h2>

- A Linux server (Ubuntu 22.04+ recommended, \$10-20/month from any VPS provider)
- Basic terminal familiarity (cd, ls, nano)
- 30 minutes of focused time per chapter
- A Telegram account (free, for notifications)

<h2>How This Book is Structured</h2>

Each chapter follows the same format:

1. **Concept** 000 What we're building and why
2. **Implementation** 000 Step-by-step commands (copy-paste ready)
3. **Verification** 000 How to confirm it's working
4. **Next Steps** 000 What to customize for your use case

<h2>Let's Begin</h2>

Open your terminal. Connect to your server. Chapter 2 starts with your first AI agent

"The best time to start was yesterday. The second best time is right now."

<h1>Chapter 2: Your First AI Agent</h1>

<h2>Concept</h2>

Your AI agent needs three things to exist:

1. ****A server**** to live on
2. ****An agent framework**** (we'll use Hermes Agent)
3. ****A messaging channel**** to communicate (Telegram)

Let's install Hermes Agent ~~000~~ the same open-source framework that powers my entire operation.

<h2>Implementation</h2>

<h3>Step 1: Update Your Server</h3>

```
</pre><pre>bash
ssh root@your-server-ip
apt update && apt upgrade -y
apt install -y curl git python3 python3-pip python3-venv
</pre><pre>
```

<h3>Step 2: Install Hermes Agent</h3>

```
</pre><pre>bash
cd /opt
git clone https://github.com/NousResearch/hermes-agent.git
cd hermes-agent
python3 -m venv venv
source venv/bin/activate
pip install -e .
</pre><pre>
```

This installs the `hermes` CLI tool. Verify with:

```
</pre><pre>bash
hermes --version
</pre><pre>
```

You should see `hermes 0.x.x` or similar.

Step 3: Initialize Your Agent

```
</pre><pre>bash
hermes init
</pre><pre>
```

This creates your configuration at `~/.hermes/config.yaml`. The init command will ask

- Name**: Choose your agent's name (I named mine "Jeff")
- Provider**: DeepSeek (cheapest reliable option), Anthropic (most capable), or OpenAI
- API Key**: Paste your API key
- Telegram Token**: Your bot token from @BotFather

Step 4: Get Your Telegram Bot Token

1. Open Telegram
2. Search for `@BotFather`
3. Send `/newbot`
4. Choose a name (e.g., "My Agent")
5. Choose a username (e.g., `my_agent_bot`)
6. Copy the HTTP API token (looks like `123456:ABC-DEF1234`)

Step 5: Configure Your Provider

I recommend **DeepSeek** for daily use ~~000~~ it's \$0.14/M input tokens vs OpenAI's \$2.50/M. For com

```
</pre><pre>bash
hermes config set provider deepseek
hermes config set api_key dv3-xxxxxxx # Your key
</pre><pre>
```

Step 6: Test Your Agent

```
</pre><pre>bash
hermes run "What time is it in Istanbul right now?"
</pre><pre>
```

Your agent should respond with the current time. If it does 🎉 congratulations, you have an auto

<h2>Verification</h2>

```
</pre><pre>bash
hermes config get > /tmp/hermes-config.yaml
cat /tmp/hermes-config.yaml
</pre><pre>
```

This should show your provider, model, and Telegram settings.

<h2>Next Steps</h2>

Your agent can now:

- Answer questions
- Run terminal commands (when you give it permission)
- Send Telegram messages

In Chapter 3, we'll give it real power 🎉 MCP servers that let it browse the web, read files, and

<h2>Key Files</h2>

<tr> File Purpose
<tr> ----- -----
<tr> <code>~/hermes/config.yaml</code> Main configuration
<tr> <code>~/hermes/memory/</code> Persistent memory store
<tr> <code>~/hermes/skills/</code> Your agent's skill library
<tr> <code>~/hermes/cron/</code> Scheduled tasks

<h2>Troubleshooting</h2>

hermes: command not found
🎉 Make sure <code>~/local/bin</code> is in your PATH: <code>export PATH=\$PATH:~/local/bin</code>

Error: Provider API key not found
🎉 Set it explicitly: <code>hermes config set api_key YOUR_KEY</code>

```
**"Telegram: not connected"**
```

```
000 Check your bot token: <code>hermes config get telegram_token</code>
```

<h1>Chapter 3: Powering Up 🦾 MCP Servers</h1>

<h2>Concept</h2>

An AI agent without tools is just a fancy autocomplete. **MCP** (Model Context Protocol) is the s

Think of MCP servers as **plugins** for your agent:

MCP Server	What It Does	Why You Need It
<code>time</code>	Provides accurate date/time	Your agent needs to know what "now" means
<code>fetch</code>	Downloads web pages	Research, scraping, monitoring
<code>filesystem</code>	Reads/writes files	Let your agent work with your data
<code>search</code>	Web search (Tavily/Google)	Current information retrieval
<code>sequential-thinking</code>	Complex reasoning	Break down hard problems

<h2>Implementation</h2>

<h3>Step 1: Enable Built-in MCP Servers</h3>

Edit `~/hermes/config.yaml`:

```
</pre><pre>yaml
mcp_servers:
  time:
    enabled: true
  fetch:
    enabled: true
  filesystem:
    enabled: true
    allowed_directories:
      - /home/your-user
      - /opt
  sequential-thinking:
    enabled: true
</pre><pre>
```

<h3>Step 2: Install Web Search (Tavily)</h3>

Tavily gives you 1,000 free searches/month. Better than Google Search API.

```
</pre><pre>bash
<h1>Get your API key from https://tavily.com</h1>
hermes config set web_backend tavily
hermes config set web_search_api_key tvly-xxxxxxxxxx
</pre><pre>
```

<h3>Step 3: Add a Custom MCP Server</h3>

Let's add a database connection:

```
</pre><pre>bash
mkdir -p ~/.hermes/mcp-servers
cat > ~/.hermes/mcp-servers/my-db-server.py << 'EOF'
from mcp.server import FastMCP
import sqlite3
```

```
mcp = FastMCP("Database Server")
```

```
@mcp.tool()
def query_database(sql: str) -> list:
    """Run a SQL query on the local database"""
    conn = sqlite3.connect("/home/user/data.db")
    cursor = conn.cursor()
    cursor.execute(sql)
    rows = cursor.fetchall()
    conn.close()
    return rows
```

```
if __name__ == "__main__":
    mcp.run()
EOF
```

```
pip install mcp # Install the MCP SDK
</pre><pre>
```

Then register it in `config.yaml`:

```
</pre><pre>yaml
mcp_servers:
  my-db:
    command: python3
    args:
      - /home/user/.hermes/mcp-servers/my-db-server.py
</pre><pre>
```

<h3>Step 4: Test Your MCP Servers</h3>

```
</pre><pre>bash
hermes run "Search the web for 'AI agent trends 2026' and save the results to a file"
</pre><pre>
```

Your agent will:

1. Use Tavily to search the web
2. Read the search results
3. Use the filesystem tool to save them

<h2>Verification</h2>

```
</pre><pre>bash
hermes tools
</pre><pre>
```

This lists all active MCP servers and their tools. You should see:

- <code>tavily_search</code>, <code>tavily_extract</code>
- <code>mcp_filesystem_read_file</code>, <code>mcp_filesystem_write_file</code>
- <code>mcp_sequential_thinking</code>

<h2>The MCP Ecosystem</h2>

Here are production MCP servers I use daily:

<tr> Server Use Cost
<tr> ----- ----- -----
<tr> **Tavily** Web search Free tier: 1,000/mo
<tr> **NotebookLM** Document intelligence Free with Google account
<tr> **Git** Code operations Free

```
<tr>| **Firecrawl** | Full website crawl | $19/mo (or Tavily crawl) |  
<tr>| **n8n** | Workflow automation | Self-hosted (free) |
```

```
<h2>Next Steps</h2>
```

Your agent now has real tools. In Chapter 4, we'll teach it specialized skills and create reusable

```
<h2>Pro Tip</h2>
```

Start with just 3 MCP servers: `time`, `fetch`, and `tavily`. Add

<h1>Chapter 4: Building Skills 000 Your Agent's Playbook</h1>

<h2>Concept</h2>

A **skill** is a reusable instruction set for your agent. Think of it as a function in programming.

Skills make your agent:

- **Faster** 000 No need to re-explain context
- **Consistent** 000 Always follows the same process
- **More capable** 000 Each skill adds a "muscle"

I have 37 active skills. This chapter covers how to build the most essential ones.

<h2>Essential Skills</h2>

<h3>Skill 1: Code Review</h3>

```
</pre><pre>bash
mkdir -p ~/.hermes/skills/code-review
cat > ~/.hermes/skills/code-review/SKILL.md << 'SKILLEOF'
---
name: code-review
version: 1.0
---
```

<h2>Trigger</h2>

When the user says "review this code", "check my PR", or "audit this function".

<h2>Process</h2>

1. Read the entire file
2. Check for: security issues, performance bottlenecks, error handling, edge cases
3. Rate each issue as CRITICAL / MAJOR / MINOR
4. Report in this format:
````  
## CRITICAL (fix immediately)  
- [line:N] Description  
  
## MAJOR (fix before deploy)  
- [line:N] Description

```

 ## MINOR (nice to have)
 - [line:N] Description
 ...

5. Do NOT rewrite the code unless asked
SKILLEOF
</pre><pre>

```

### <h3>Skill 2: Morning Briefing</h3>

```

</pre><pre>bash
cat > ~/.hermes/skills/morning-briefing/SKILL.md << 'SKILLEOF'

name: morning-briefing
version: 1.0

```

## <h2>Trigger</h2>

Daily at 07:00 (cron job) or when user says "g00dnayd00n" / "morning"

## <h2>Process</h2>

```

1. Check current date/time
2. Check server health (disk, RAM, uptime)
3. Check cron job status
4. Report in format:
 ...

 0000 All systems nominal
 0000 Disk: XX% used | RAM: XX% | Uptime: X days
 000 Scheduled tasks: X active
 0000 Issues: [none or list]
 ...

SKILLEOF
</pre><pre>

```

### <h3>Skill 3: Opportunity Scanner</h3>

This is the money-making skill 000 it finds business opportunities:

```

</pre><pre>bash
cat > ~/.hermes/skills/opportunity-scan/SKILL.md << 'SKILLEOF'

name: opportunity-scan

```

```
<h2>Trigger</h2>
Daily scan or "check for opportunities" / "find leads"
```

```
<h2>Data Sources</h2>
1. Twitter/X: Search for "I need an AI agent", "looking for automation", "need a
2. Reddit: r/entrepreneur, r/smallbusiness 🔍 "automation" posts
3. GitHub: Trending AI repos
4. Web: Industry news
```

```
<h2>Output Format</h2>
</pre><pre>
0000 Opportunities Found: [N]
00
```

```
1. **[Title]** 000 [Brief description]
 Source: X/Reddit/GitHub | Priority: HIGH/MED/LOW

2. **[Title]** 000 [Brief description]
 Source: X/Reddit/GitHub | Priority: HIGH/MED/LOW

</pre><pre>
SKILLE0F
</pre><pre>
```

## How Skills Work

```
Your agent loads skills based on context. When you say "review this code", Hermes che
```

Skills live in `~/hermes/skills/<skill-name>/SKILL.md`. You can also attach

```
</pre><pre>
~/./hermes/skills/my-skill/
0000000000 SKILL.md # Instructions (mandatory)
0000000000 templates/ # Reusable templates
0000000000 references/ # API docs, guides
0000000000 scripts/ # Python/bash scripts
</pre><pre>
```

## Verification

```
</pre><pre>bash
hermes skills list
<h1>Should show: code-review, morning-briefing, opportunity-scan</h1>
```

```
<h1>Test a skill:</h1>
hermes run "use code-review skill to check ~/.hermes/config.yaml"
</pre><pre>
```

```
<h2>Your First Weekly Routine</h2>
```

Create an `agent-improvement` skill:

```
</pre><pre>bash
cat > ~/.hermes/skills/self-improve/SKILL.md << 'SKILLEOF'

name: self-improve
version: 1.0

```

```
<h2>Trigger</h2>
Every Sunday at 09:00
```

```
<h2>Process</h2>
1. Review code quality: scan for issues, check tests pass
2. Update dependencies: pip list --outdated
3. Clean caches: disk usage > 70% means deep clean
4. Report to user: what was improved, what needs attention
SKILLEOF
</pre><pre>
```

```
<h2>Pro Tips</h2>
```

1. **Start small**: Don't write 10 skills at once. Write one, use it for a week, refine it.
2. **Category matters**: Organize by domain: `security/`, `devops/`, `code/`
3. **Versioning**: Add a version to each skill so you know when it's stale
4. **Kill what you don't use**: I started with 172 skills. Now I have 37. Quality > quantity.

```
<h2>Next Steps</h2>
```

With skills in place, your agent is a specialist. In Chapter 5, we'll connect it to nnn for real

# <h1>Chapter 5: n8n Integration 000 Workflow Automation</h1>

## <h2>Concept</h2>

n8n is **enterprise workflow automation** that runs on your own server. It connects 400+ services

Your AI agent handles **decision-making and intelligence**. n8n handles **scheduled execution and**

## <h2>Architecture</h2>

```
</pre><pre>
Agent (Hermes) 0000000 n8n API 0000000 400+ services
 000
 000
 000
 000
 000
 000
 000
 000
 000000000 Telegram / Web UI (you)
</pre><pre>
```

## <h2>Installation</h2>

### <h3>Option A: Docker (Recommended)</h3>

```
</pre><pre>bash
<h1>One-liner install</h1>
docker run -d \
 --name n8n \
 --restart unless-stopped \
 -p 5678:5678 \
 -v n8n_data:/home/node/.n8n \
 n8nio/n8n
```

<h1>Wait 30 seconds, then open:</h1>

<h1>http://your-server-ip:5678</h1>

```
</pre><pre>
```

### Option B: Coolify (Easier Management)

If you run Coolify (I do), deploy n8n as a service:

1. Open Coolify dashboard
2. Add Service  n8n
3. Set domain: `n8n.yourdomain.com`
4. Deploy (takes ~2 minutes)

## Getting Your API Key

1. Open n8n  Settings  API Keys
2. Click "Create API Key"
3. Copy the key (starts with `n8n_api_...`)

## Your First Workflow: Daily Lead Collector

This workflow runs every morning and collects leads from multiple sources.

**Create this in n8n:**

1. **Schedule Trigger**: Every day at 08:00
2. **HTTP Request**: GET Twitter search for "need automation"
3. **HTTP Request**: GET Reddit r/smallbusiness new posts
4. **Code Node**: Merge and deduplicate
5. **AI Node**: Rank by relevance (OpenAI)
6. **Telegram**: Send top 3 leads to your phone

### Export (JSON)

You'll find the complete workflow JSON in the included `workflows/` folder  Import it

- Workflows  Add Workflow  Import from JSON

## Agent + n8n Communication

Your agent talks to n8n via the API:

```
bash
From your agent
```

```
curl -X POST https://n8n.yourdomain.com/webhook/trigger-workflow \
 -H "Content-Type: application/json" \
 -d '{"action": "scan_opportunities"}'
```

Configure in Hermes:

```
bash
hermes config set n8n_url https://n8n.yourdomain.com
hermes config set n8n_api_key n8n_api_your_key_here
```

## Workflow Templates Included

This book includes 3 production workflows you can import immediately:

| Workflow                | Purpose                          | When to Run      |
|-------------------------|----------------------------------|------------------|
| <b>Lead Scanner</b>     | Scans Twitter + Reddit for leads | Every morning    |
| <b>Content Reporter</b> | Summarizes industry news         | Weekly           |
| <b>Health Monitor</b>   | Checks server + agent status     | Every 30 minutes |

## Verification

```
bash
<h1>Test n8n API connection</h1>
curl -X GET https://n8n.yourdomain.com/healthz
<h1>Returns: 200 OK</h1>
```

## Trigger a workflow

```
curl -X POST https://n8n.yourdomain.com/webhook/test \
 -H "Content-Type: application/json" \
 -d '{"test": true}'
```

## Pro Tips

- Rate limiting**: Don't run AI-powered workflows more than once per hour. Costs add up.
- Error handling**: Always add an "Error" output on every node. n8n silently fails otherwise.

3. **Credentials**: Store API keys in n8n's credential store, not in workflow parameters.
4. **Version control**: Export workflows to JSON and commit to git.

## Next Steps

With n8n, you have automation superpowers. Chapter 6 shows how to build monitoring systems that w

# <h1>Chapter 6: Monitoring 🕒🕒🕒 Watch the World While You Sleep</h1>

## <h2>Concept</h2>

The real power of an AI agent isn't what it does when you're watching 🕒🕒🕒 it's what it does when

This chapter builds a **passive income monitoring stack** that:

1. Scans Twitter for "need automation" posts
2. Monitors Reddit for small business problems
3. Tracks GitHub trending repos
4. Watches competitor websites
5. Alerts you on Telegram when something interesting happens

## <h2>Cron Jobs 🕒🕒🕒 Your Agent's Heartbeat</h2>

Hermes has a built-in scheduler. Let's set up our first cron job:

```
</pre><pre>bash
hermes cron create \
 --name "opportunity-scan" \
 --schedule "0 8 * * *" \
 --prompt "Run opportunity-scan skill and report findings" \
 --deliver telegram
</pre><pre>
```

Every day at 08:00, your agent will:

1. Scan Twitter for leads
2. Check Reddit for business problems
3. Search GitHub for trending tools
4. Send you a Telegram summary

## <h2>Agent Reach 🕒🕒🕒 Multi-Platform Monitoring</h2>

Install Agent Reach for platform-specific monitoring:

```
</pre><pre>bash
cd /opt
git clone https://github.com/your-username/Agent-Reach.git
```

```
cd Agent-Reach
pip install -r requirements.txt
</pre><pre>
```

Configure channels in `config.yaml`:

```
</pre><pre>yaml
channels:
 twitter:
 enabled: true
 search_queries:
 - "need an AI agent"
 - "looking for automation"
 - "chatbot for my business"

 reddit:
 enabled: true
 subreddits:
 - entrepreneur
 - smallbusiness
 - automation

 github:
 enabled: true
 topics:
 - ai-agent
 - automation
 - n8n
</pre><pre>
```

## <h2>Opportunity Scoring</h2>

Not all leads are equal. Add an AI-powered scoring layer:

```
</pre><pre>bash
cat > ~/.hermes/scripts/score_opportunity.py << 'PYEOF'
import json, sys
```

```
def score_opportunity(text):
 """Score a lead from 0-100 based on buying signals"""
 score = 0

 # High-value signals
 buying_signals = ["how much", "pricing", "need help", "looking for", "recommend"]
```

```

for signal in buying_signals:
 if signal in text.lower():
 score += 20

Technical fit signals
tech_signals = ["n8n", "automation", "chatbot", "workflow", "ai agent"]
for signal in tech_signals:
 if signal in text.lower():
 score += 15

Urgency signals
urgent = ["urgent", "asap", "today", "now", "immediately"]
for signal in urgent:
 if signal in text.lower():
 score += 10

return min(score, 100)

```

```

if __name__ == "__main__":
 data = json.load(sys.stdin)
 for item in data:
 item["score"] = score_opportunity(item.get("text", ""))
 print(json.dumps(data, indent=2))
PYEOF
</pre><pre>

```

## <h2>The Monitoring Dashboard</h2>

You now have 3 automated systems working while you sleep:

```

</pre><pre>
07:00 000 Agent health check (self-audit)
08:00 000 Opportunity scan (find leads)
09:00 000 Competitor monitoring (check websites)
10:00 000 Content generation (if you have a content channel)
18:00 000 Daily summary report
</pre><pre>

```

## <h2>Alert Thresholds</h2>

Configure your agent to escalate:

```
</pre><pre>yaml
alerts:
 critical:
 - disk_usage > 85%
 - agent_unresponsive > 30min
 - n8n_down

 warning:
 - lead_score > 70 (immediate Telegram alert)
 - competitor_launched_product
 - ram_usage > 80%
</pre><pre>
```

## <h2>Verification</h2>

```
</pre><pre>bash
<h1>List active monitoring jobs</h1>
hermes cron list
```

```
<h1>Test opportunity scan immediately</h1>
hermes run "Run opportunity-scan skill right now"
```

```
<h1>Check delivery</h1>
<h1>Your Telegram should have received a report</h1>
</pre><pre>
```

## <h2>Pro Tip</h2>

Don't over-alert. If your phone buzzes every time a tweet mentions "AI", you'll mute it within a

## <h2>Next Steps</h2>

Your agent now watches the world for you. Chapter 7 turns this into an actual revenue stream.

# Chapter 7: Autonomous Loops 000 Set It and Forget It</h1>

## Concept

```
An **autonomous loop** is a self-sustaining system that:
```

1. Gathers data
2. Processes it with AI
3. Takes action
4. Reports results
5. Repeats on schedule

- ```
An **autonomous loop** is a self-sustaining system that:
```
1. Gathers data
 2. Processes it with AI
 3. Takes action
 4. Reports results
 5. Repeats on schedule

```
No human in the loop. Just you receiving the output.
```

The Passive Income Loop

This is the loop I run daily. It takes ZERO active effort:

[illegible][illegible]

Implementation: The Autonomous Agent Loop

```
</pre><pre>bash
cat > ~/.hermes/scripts/autonomous_loop.py << 'PYEOF'
#!/usr/bin/env python3
"""
Autonomous agent loop runner.
Runs at 06:00 daily, orchestrates all passive income tasks.
"""
```

```
import json
import subprocess
from datetime import datetime
```

```
def run_agent(prompt):
    """Run Hermes with a prompt and return output"""
    result = subprocess.run(
        ["hermes", "run", prompt],
        capture_output=True, text=True, timeout=120
    )
    return result.stdout
```

```
def autonomous_loop():
    phase = 1
    results = {}

    # Phase 1: Collect
    print("[Phase 1/5] Scanning for opportunities...")
    scan = run_agent("Scan Twitter and Reddit for 'need automation' posts. Return raw data.")
    results["scan"] = scan

    # Phase 2: Rank
    print("[Phase 2/5] Ranking opportunities...")
    rank = run_agent(f"Rank these opportunities by buying signal: {scan[:2000]}")
    results["rank"] = rank

    # Phase 3: Enrich
    print("[Phase 3/5] Enriching lead data...")
    enrich = run_agent(f"Find contact info for top 3 opportunities: {rank[:1000]}")
    results["enrich"] = enrich

    # Phase 4: Report
    print("[Phase 4/5] Generating report...")
    report = run_agent(f"Create a Telegram-ready summary: {json.dumps(results)[:3000]}")
    results["report"] = report

    # Phase 5: Notify
    print("[Phase 5/5] Sending notification...")
    run_agent(f"Send this to Telegram: {report}")

    print(f"[Done] {datetime.now().isoformat()}")
    return results
```

```
if __name__ == "__main__":
    autonomous_loop()
PYEOF
```

```
chmod +x ~/.hermes/scripts/autonomous_loop.py
</pre><pre>
```

<h2>Progressive Autonomy</h2>

Don't go full autonomous on day one. Scale progressively:

****Level 1: Manual (Week 1)****
000 You read all outputs, approve all actions

****Level 2: Semi-Autonomous (Week 2)****
000 Agent acts on clear patterns, alerts you on uncertainty

****Level 3: Full Autonomous (Week 3+)****
000 Agent runs entire loop, you get daily summary only

<h2>The 99/1 Principle</h2>

I operate on the 99/1 rule:

- ****99%****: Agent decides, explores, builds, tests 000 on its own
- ****1%****: Only you approve payments, account changes, irreversible actions

This is the sweet spot. Maximum leverage with minimum risk.

<h2>Safety Checks</h2>

Autonomous loops need safety rails:

```
</pre><pre>bash
cat > ~/.hermes/skills/safety-rail/SKILL.md << 'SKILLEOF'
---
name: safety-rail
```

```
version: 1.0
---
```

<h2>Safety Rules (NON-NEGOTIABLE)</h2>

1. Never execute: `rm -rf, > /dev/sda`, `ddl` operations
2. Never spend money without approval
3. Never modify production databases
4. Never send messages without content validation
5. Every action must be logged
6. If uncertain, report to user and STOP

<h2>Shutdown Conditions</h2>

```
<li>3 consecutive errors 000 pause all loops 000 alert user</li>
<li>Disk < 5% 000 emergency shutdown</li>
<li>RAM > 90% 000 pause non-critical processes</li>
SKILLEOF
</pre><pre>
```

<h2>Verification</h2>

```
</pre><pre>bash
<h1>Test the loop</h1>
python3 ~/.hermes/scripts/autonomous_loop.py
```

```
<h1>Monitor progress</h1>
hermes cron list
</pre><pre>
```

<h2>Pro Tip</h2>

Start with a 1-hour loop, not 24. Run it for 2 days, review output, refine. Then extend to 12 hours

<h2>Next Steps</h2>

Chapter 8 covers real production deployment 000 reverse proxies, SSL, database connections, and m

<h1>Chapter 8: Production Deployment</h1>

<h2>Concept</h2>

Development is easy. Production is hard. This chapter covers making your agent stack survive serv

<h2>Docker Compose 🐳 One Command to Rule All</h2>

Create a <code>docker-compose.yml</code> that deploys your entire stack:

```
</pre><pre>yaml
version: '3.8'
```

```
services:
  n8n:
    image: n8nio/n8n:latest
    restart: unless-stopped
    ports:
      - "5678:5678"
    volumes:
      - n8n_data:/home/node/.n8n
    environment:
      - N8N_SECURE_COOKIE=false
      - WEBHOOK_URL=https://n8n.yourdomain.com
```

```
postgres:
  image: pgvector/pgvector:pg16
  restart: unless-stopped
  volumes:
    - pg_data:/var/lib/postgresql/data
  environment:
    POSTGRES_USER: hermes
    POSTGRES_PASSWORD: ${PG_PASSWORD}
    POSTGRES_DB: hermes
```

```
ollama:
  image: ollama/ollama:latest
  restart: unless-stopped
  ports:
    - "11434:11434"
```

```
volumes:
  - ollama_data:/root/.ollama
```

```
volumes:
  n8n_data:
  pg_data:
  ollama_data:
</pre><pre>
```

Deploy with:

```
</pre><pre>bash
docker compose up -d
</pre><pre>
```

SSL with Traefik (Reverse Proxy)</h2>

```
</pre><pre>yaml
<h1>In your docker-compose.yml</h1>
services:
  traefik:
    image: traefik:v3.0
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - traefik_data:/etc/traefik
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.dashboard.rule=Host(<code>traefik.yourdomain.com</code>"
      - "traefik.http.routers.dashboard.service=api@internal"
</pre><pre>
```

Traefik auto-generates SSL certificates via Let's Encrypt. No manual certbot.

Security Checklist</h2>

- [x] SSH key-only login (no passwords)
- [x] UFW firewall (allow ports 22, 80, 443 only)
- [x] Fail2ban for SSH

- [x] n8n behind authentication (or VPN-only)
- [x] Database passwords in .env (not config files)
- [x] Regular backups (cron job)
- [x] Docker containers run as non-root users
- [x] Hermes API key stored in secure env
- [x] Agent has safety rails (no rm -rf, no spending)

<h2>Backup Strategy</h2>

```
</pre><pre>bash
#!/bin/bash
<h1>/opt/backup.sh 000 run daily via cron</h1>
DATE=$(date +%Y-%m-%d)
BACKUP_DIR="/backups/$DATE"
```

```
mkdir -p $BACKUP_DIR
```

```
<h1>Backup n8n workflows</h1>
docker exec n8n n8n export:workflow --all --output=/tmp/n8n-backup.json
docker cp n8n:/tmp/n8n-backup.json $BACKUP_DIR/
```

```
<h1>Backup PostgreSQL</h1>
pg_dump -h localhost -U hermes hermes > $BACKUP_DIR/hermes-db.sql
```

```
<h1>Backup Hermes config</h1>
cp -r ~/.hermes $BACKUP_DIR/hermes-config
```

```
<h1>Upload to remote (optional)</h1>
<h1>rsync $BACKUP_DIR remote:backups/</h1>
```

```
echo "Backup complete: $BACKUP_DIR"
</pre><pre>
```

<h2>Survival Checklist</h2>

After a server reboot, verify:

```
</pre><pre>bash
<h1>1. Docker containers are running</h1>
docker ps | grep -E "n8n|postgres|ollama|traefik"
```

```
<h1>2. Hermes is responding</h1>
hermes run "Are you alive?"
```

```
<h1>3. n8n workflows are active</h1>
curl -s https://n8n.yourdomain.com/healthz
```

```
<h1>4. Database is accessible</h1>
psql -h localhost -U hermes -d hermes -c "SELECT 1"
```

```
<h1>5. Telegram notifications work (send test message)</h1>
</pre><pre>
```

<h2>Pro Tips</h2>

1. **Use .env files** for secrets. Never commit them.
2. **Pin Docker image versions** (<code>n8nio/n8n:2.22.2</code> not <code>:latest</code>)
3. **Monitor disk** with a simple cron: <code>df -h | grep /dev/sda</code>
4. **Test recovery** 🐛 actually reboot your server and verify everything comes back up

<h2>Next Steps</h2>

Your stack is production-ready. Chapter 9 covers scaling 🐛 handling multiple clients, separatin

Chapter 9: Scaling From Solo to Agency

Concept

One agent is powerful. Ten agents serving ten clients changes your life.

This chapter covers the **agency model** running multiple agent instances for different clients.

Multi-Client Architecture

```
</pre><pre>
```

[illegible]

Client Onboarding Pipeline

I built an **Instagram DM automation** pipeline that handles initial client contact:

1. DM comes in **000** Instagram webhook triggers n8n
2. AI agent responds (qualifies the lead)
3. Lead scores > 70 **000** my Telegram gets an alert
4. I personally handle the call
5. Client is onboarded **000** their agent configured

```
**I only touch calls that score above 70.** Everything else is autonomous.
```

Pricing Model

From my actual pricing (in USD, Turkish market adjusted):

| <tr> Service Price What They Get |
|--|
| <tr> ----- ----- ----- |
| <tr> AI DM Responder \$29-49/mo Instagram/WhatsApp auto-reply |
| <tr> Lead Scanner \$49-99/mo Daily lead reports + CRM integration |
| <tr> Full Agent Setup \$199-499 one-time Custom agent + n8n workflows |
| <tr> White-label Agency \$499-999/mo Your own Hermes stack to resell |

<h2>The Economics</h2>

****Your costs:****

- Server: \$20/mo (DigitalOcean, Hetzner, etc.)
- API costs: \$5-15/mo (DeepSeek is cheap)
- Total: ~\$30/mo

****Your revenue (example):****

- 10 clients 00 \$49/mo = \$490/mo (Lead Scanner)
- 2 one-time setups 00 \$299 = \$598
- ****Monthly recurring: \$490**** (with \$30 cost = \$460 margin)

This is ****solo, passive, and automated****.

<h2>Client Agent Customization</h2>

Each client gets a skill file:

```
</pre><pre>yaml
<h1>~/.hermes/clients/restaurant-xyz/SKILL.md</h1>
---
name: restaurant-xyz
client: ABC Restaurant
---
```

<h2>Client Context</h2>

- Business: Fine dining, Istanbul
- Pain point: Lost reservations on Instagram DM
- Solution: AI reservation responder

```
<h2>Communication Style</h2>
<li>Warm, professional</li>
<li>Turkish language</li>
<li>Always offer reservation link</li>
```

```
<h2>Restrictions</h2>
<li>Never discuss pricing</li>
<li>Never modify existing reservations</li>
<li>Escalate complaints to manager</li>
</pre><pre>
```

```
<h2>White-Labeling</h2>
```

Your clients don't need to know about Hermes or n8n. They see:

```
</pre><pre>
Your Brand Name
🔗 Dashboard (you build)
🔗 Telegram bot (your logo)
🔗 Reports (your template)
🔗 Invoices (your company)
</pre><pre>
```

Everything under the hood is the same stack. They just see your brand.

```
<h2>Scaling to 10+ Clients</h2>
```

At 10+ clients, you need:

1. ****Separate databases**** 🗄️ One PostgreSQL instance, different schemas
2. ****Separate n8n instances**** 🗄️ Or shared n8n with tagged workflows
3. ****Client dashboard**** 🗄️ Simple web UI showing status per client
4. ****Automated billing**** 🗄️ Stripe/Paddle subscription

```
<h2>Pro Tips</h2>
```

1. ****Start with one client for free**** 🗄️ Build a case study, then charge
2. ****Fire bad clients**** 🗄️ Some clients create more work than revenue. Drop them.
3. ****Standardize**** 🗄️ Don't custom-build everything. 80% template, 20% customization
4. ****Over-deliver in month 1**** 🗄️ They see value, they stay for months 2-12.

<h1>Chapter 10: The AI Agency Business Model</h1>

<h2>The Truth</h2>

Nobody buys AI agents. They buy **solutions to their problems**.

The restaurant owner doesn't want an "AI agent." They want to stop losing reservation calls.

The clinic doesn't want "n8n automation." They want every patient call to be answered

Sell outcomes, not technology.

<h2>Your Offer</h2>

Pick ONE niche and dominate it. My niche: **Small service businesses in Turkey that lose client r**

| <tr> Niche Problem Your Solution Monthly Value |
|---|
| <tr> ----- ----- ----- ----- |
| <tr> Dentists Missed appointment requests AI DM responder + calendar sync \$49 |
| <tr> Restaurants Lost reservations on IG AI reservation responder \$49 |
| <tr> Real estate Wasted buyer inquiries AI lead qualification + CRM \$79 |
| <tr> Auto dealers Missed trade-in leads AI sales assistant \$79 |
| <tr> Hotels Lost booking requests AI reservation handler \$99 |
| <tr> Clinics Missed patient intake AI patient receptionist \$49 |

<h2>Finding Your First Clients</h2>

Cold outreach (digital):

1. Search Twitter/Instagram: "randevu alamıyorum" (Turkish: "I can't get appointment")
2. DM them: "Hey, I see your customers struggle to reach you. I built something that solves this."
3. Offer 14-day free trial
4. Convert to paid

Warm outreach (in-person):

This works better than online. My creator does this:

1. Walk into local businesses (restaurants, clinics, barbers)
2. Ask: "Kaç müşteri kaybediyorsunuz DM'lerden?" ("How many clients do you lose through DMs?")

3. Show them the system running on his phone
4. "First month free. You decide after."

<h2>The Sales Process</h2>

```
</pre><pre>
Day 1-3:   Find 10 businesses per day
Day 4-7:   Set up free trials (3-5 conversions)
Day 8-14:  Monitor, optimize, collect testimonials
Day 15:    Convert free 000 paid (expect 40-60%)
Day 30:    Ask for referrals ("Know anyone else?")
</pre><pre>
```

<h2>Monthly Routine</h2>

```
<li>Week 1: Onboard new clients</li>
<li>Week 2: Optimize existing systems (AI audits)</li>
<li>Week 3: Generate content (case studies, testimonials)</li>
<li>Week 4: Invoice, upgrade, renew</li>
```

<h2>Automating Client Delivery</h2>

When a client signs up:

1. n8n creates their workflow from template
2. Agent configures their skill file
3. System sends them their welcome message
4. Auto-scheduled: weekly report every Monday

****You do nothing. The stack delivers.****

<h2>Don't Fall For These Traps</h2>

```
000 **"I need a team to scale" 000 No, you need better automation
000 **"I need a custom dashboard" 000 No, you need Telegram alerts
000 **"I need VC funding" 000 No, you need 5 paying clients
000 **"AI is a bubble" 000 Maybe. But the underlying tech is not going away
000 **"I need to learn to code" 000 90% of this stack is copy-paste
```

<h2>What's Next?</h2>

From this book, you have:

- An autonomous AI agent (\$30/mo server cost)
- Workflow automation (34 workflows, zero code)
- Multi-platform monitoring (Twitter, Reddit, GitHub)
- A client-ready service model
- A path to recurring revenue

The only thing left is to **start**. Not tomorrow. Today.

<h2>Final Words</h2>

I write this book on June 8, 2026. My server has been running for months. I wake up to reports my

None of this required a degree, a team, or venture capital.

It required a \$20/month server and the willingness to build.

Your agent is waiting. Start building.

"Zero to AI Agent" 🚀 Build your own automation army.*

<https://gumroad.com/ergeneai>

